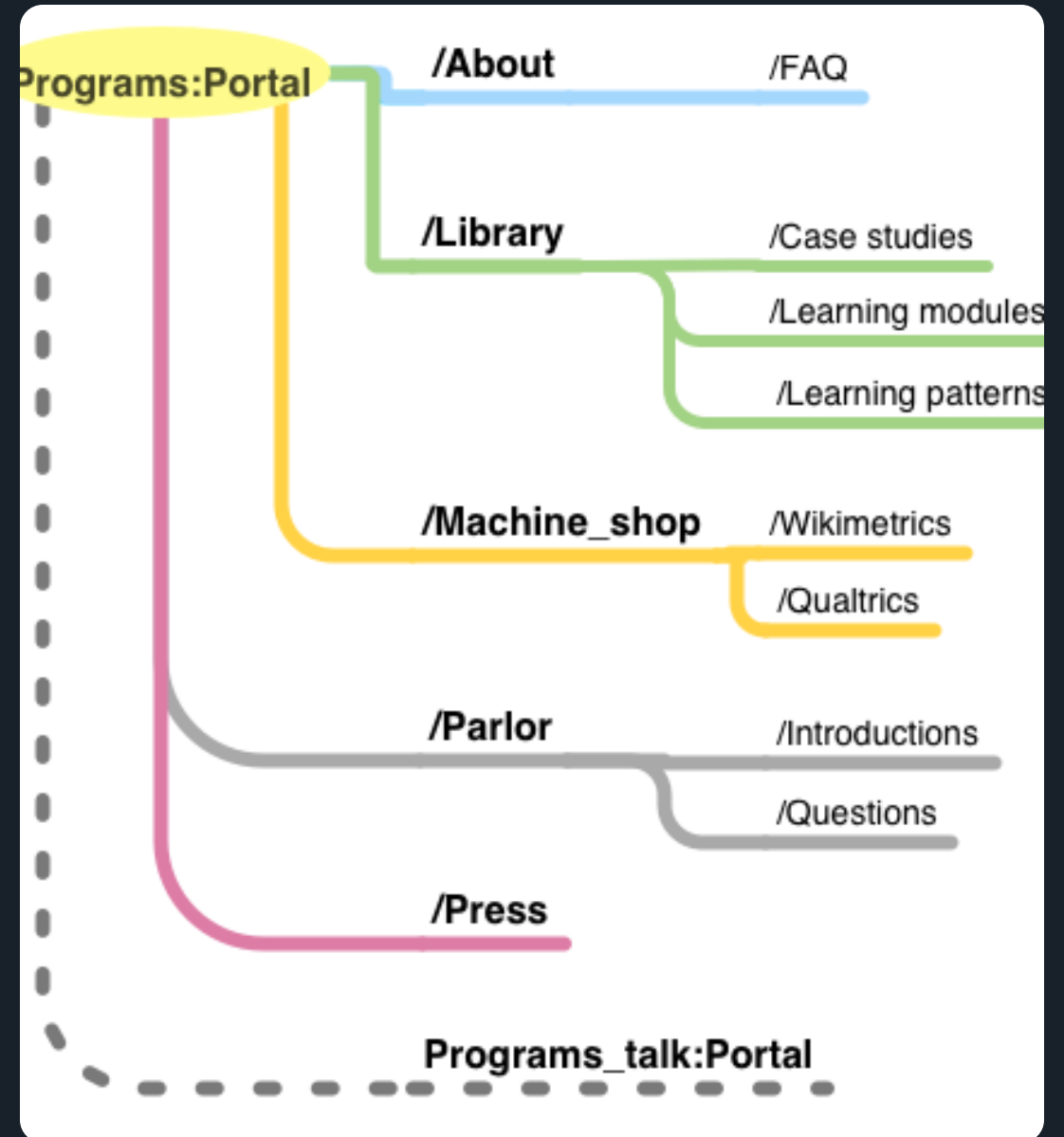


Content Systems and Multi-Page Sites

Planning an information architecture, a predictable navigation pattern, and shared page templates so a small site stays coherent as it grows from one page to many.



This week's outcomes

By the end of this session you can plan, structure and wire navigation for a small multi-page site.

Plan

Turn a content inventory into a grouped information architecture using a real card-sorting method.

Structure

Choose a site-map topology and a URL scheme that stay predictable as pages are added.

Build

Implement one shared navigation pattern across at least three real HTML pages using a page-shell template.

What information architecture actually is

IA is the structure a visitor infers before they read a single sentence.

Information architecture (IA) is the practice of deciding what content exists, how it is grouped, and how a visitor moves between groups — before any visual design happens.

The IA question

For every page: what is this content, which group does it belong to, and what is one click away from it?

Why it matters here

A site with a clean visual system but a confusing structure still fails — the reader can't find the page that answers their question.

Step 1: content inventory

List every page and fragment the site needs before grouping anything.

- 1 List every planned page and any repeating content block (e.g. a testimonial, a contact block).
- 2 Note each item's purpose in one sentence — what task does this page complete for a visitor?
- 3 Flag near-duplicates: two pages answering the same question should usually merge into one.
- 4 Flag orphans-in-waiting: any item with no obvious page to link from yet.

A content inventory is a spreadsheet, not a design — rows are pages, columns are purpose, audience and status.

Step 2: group by task, not by org chart

The most common IA mistake is grouping by who made the content, not what the visitor is trying to do.

Approach

Path shape

Consequence

Organisation-first grouping

“About → Departments → Marketing → Our team”

Matches how the organisation sees itself, not how a visitor searches

Task-first grouping

“Work → Case studies → [project]”

Matches the question a visitor actually arrived with

Rule of thumb: name each group after the question it answers, not after the team that owns it.

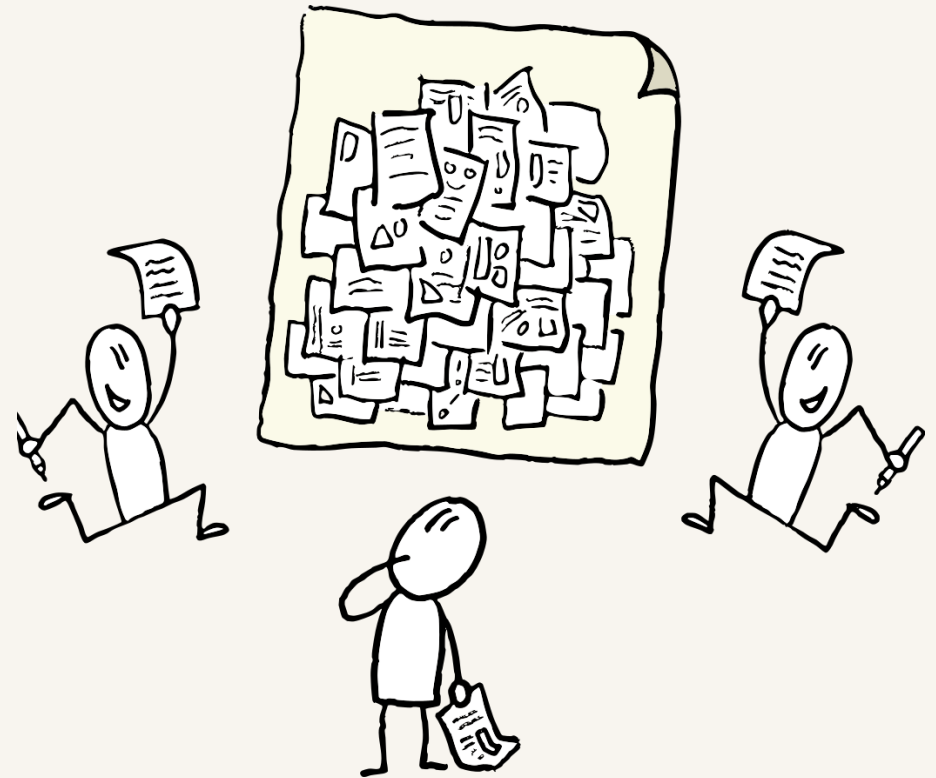
Card sorting: a real technique, not a guess

Ask people to group your actual content — don't decide the grouping alone at a desk.

Write each planned page or content item on its own card. Ask 3-5 people, individually, to sort the cards into groups that make sense to them and name each group.

Open card sort

Closed card sort



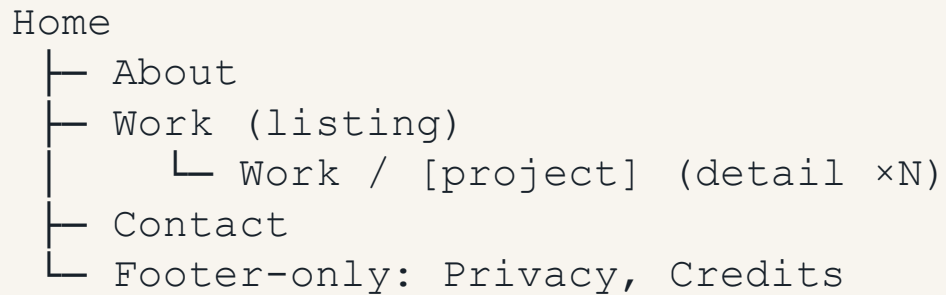
Site-map topologies: three real shapes

Every small site's structure is one of these, or a deliberate mix.

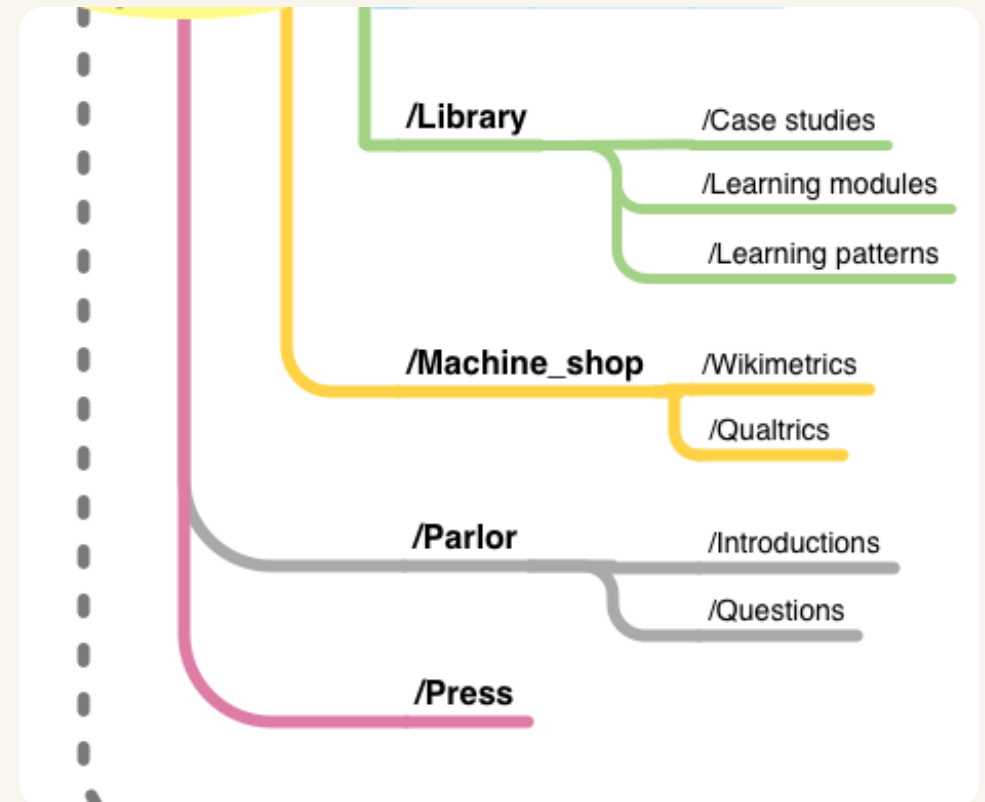
Topology	Shape	Small-site example
Hierarchical	A trunk page branches into sections, each into sub-pages	A documentation site: Home → Guides → Setup → Install
Flat	Every page sits one click from every other, no sections	A 4-page portfolio: Home, About, Work, Contact
Hub-and-spoke	One hub page links out to many independent destinations, each returns to the hub	A directory site linking to member profile pages

A worked site map: small business site

Home, about, work, contact, plus two secondary pages — a realistic Assignment 4 scale.



Every page above the footer row has an inbound link from the primary nav — nothing is reachable only by guessing a URL.



Primary navigation: the contract with every page

The same links, in the same order, in the same place — on every single page.

```
<header>
  <nav aria-label="Primary">
    <ul>
      <li><a href="/">Home</a></li>
      <li><a href="/about/">About</a></li>
      <li><a href="/work/" aria-
current="page">Work</a></li>
      <li><a href="/contact/">Contact</a></li>
    </ul>
  </nav>
</header>
```

Landmark role

A nav element identifies this as a navigation region — assistive tech can jump straight to it.

aria-current="page"

Marks which link matches the current page, so the active state is announced, not just coloured.

Breadcrumbs: where am I, not where can I go

A breadcrumb trail names the path from home to the current page — it does not replace primary nav.

```
<nav aria-label="Breadcrumb">
  <ol>
    <li><a href="/work/">Work</a></li>
    <li aria-current="page">Portfolio site</li>
  </ol>
</nav>
```

Use breadcrumbs only once a site is at least two levels deep — on a flat 4-page site they add noise, not orientation.



Local nav and footer nav: the other two patterns

Primary nav is not the only navigation a page needs.

Pattern	Job	Typical use
Local / in-page nav	Move within one long page or one section (e.g. a table of contents)	A single, sectioned reference page
Footer nav	Catch secondary pages that don't belong in primary nav (legal, credits, sitemap link)	Privacy policy, source attribution page

Every page a footer links to must also be reachable another way — a footer-only page with no other inbound link is still an orphan.

Consistent placement is an accessibility requirement

Predictable navigation location is not a style choice — it's tested behaviour (WCAG 3.2.3).

Consistent navigation

Repeated navigation components must occur in the same relative order on every page they appear — a keyboard user builds a mental map of tab order.

Skip link

A visually-hidden-until-focused link that jumps a keyboard user straight past repeated nav to <main>, so it isn't tabbed through on every page.

```
<a class="skip-link" href="#main">Skip to main content</a>
...
<main id="main">...</main>
```

The page shell: what repeats vs what's unique

Every page is one shared shell wrapped around one unique content region.

header + nav + footer are the shell — identical markup, identical classes, on every page. <main> holds the one thing that page is actually for.

The shell never changes

Frames

cols, rows, Verschachtelung

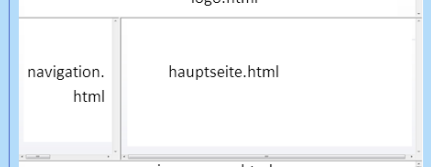
```
<frameset cols="30%,40%,30%">
  <frame src="links.html" />
  <frame src="mitte.html" />
  <frame src="rechts.html" />
</frameset>
```



```
<frameset rows="30%,40%,30%">
  <frame src="oben.html" />
  <frame src="mitte.html" />
  <frame src="unten.html" />
</frameset>
```



```
<frameset rows="15%,*,10%">
  <frame src="logo.html" name="logo" />
  <frameset cols="25%,75%">
    <frame src="navigation.html" name="navi" />
    <frame src="hauptseite.html" name="anzeige" />
  </frameset>
  <frame src="impressum.html" name="impressum" />
</frameset>
```



Keeping the shell consistent by hand

Without a framework, discipline replaces automation — name the technique so it's checkable.

- 1 Write the header/nav/footer markup once, in one file, and treat it as the source of truth.
- 2 Copy that exact block into every page — never retype it from memory.
- 3 Give shell elements stable, descriptive classes (site-header, site-nav, site-footer) reused verbatim on every page.
- 4 After any shell edit, diff every page's shell markup against the source file before publishing.

This is a lightweight, hand-built version of what a templating system or static-site generator automates — same idea, manual enforcement.

Two reusable page patterns: listing and detail

Almost every content site reduces to these two page types, repeated.

Listing page

Shows many items at a glance — a project gallery, a blog index, a product grid. Each entry links to its own detail page. The template is the same; only the item data changes per entry.

Detail page

Shows one item in full — a single blog post, a single project. Every detail page follows the same section order so a returning visitor already knows where to look.

A content model: the fields every detail page needs

Decide the fields before writing a single page — this is a hand-built version of a CMS content model.

Field	Type	Rule
title	Text	Becomes the h1 and the browser tab title
hero image	Image + alt text	One image, with alt text written for its role on this page
summary	1-2 sentences	Used on the listing page AND as the page's meta description
body	Structured HTML	Headings, paragraphs, lists — the unique content itself
link / call to action	URL + label	Where this item sends the visitor next

Why a content model prevents drift

Without an explicit model, each new page quietly reinvents its own structure.

Page 3 of a growing site is fine on instinct. Page 12 is where undocumented structure breaks — one project page has no image, another buries its link in a paragraph, a third has two headings labelled h1.

The fix

Write the field list down once (a text file is enough) and check each new page against it before publishing.

The test

Could you swap this page's body text into another page of the same type without touching the layout?

URL structure: predictable and human-readable

A URL is part of the interface — a visitor reads it before the page even loads.

URL	Problem	Risk
<code>/index.html?p=7&cat=3</code>	Meaningless once shared; breaks if the numbering changes	High
<code>/blog/post-7/</code>	Readable but still arbitrary — tells you nothing about content	Medium
<code>/work/portfolio-redesign/</code>	Matches the nav label and the page's own heading exactly	Low
Keep the URL segment, the nav label, and the on-page <h1> in agreement — three different names for one page is a navigation bug.		

The orphan page: reachable by URL, invisible in nav

The most common real defect a scaled-up hand-built site develops.

An orphan page loads fine if you type its exact URL — but no nav link, breadcrumb, or footer link points to it. A visitor, and a search engine, can never discover it.

How it happens

Copyright 2012 by the author(s). All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or by any information storage and retrieval system, without permission in writing from the author(s).

The check

Copyright 2012 by the author(s). All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or by any information storage and retrieval system, without permission in writing from the author(s).



What breaks first when a site scales past a handful of pages

The same three failures, in order, every time a site grows without a system.

Inconsistent navigation

Page 6 has a link the others don't; visitors lose the mental map week 8 taught you to protect

Duplicated header/footer code

A copy-pasted shell drifts — one page's markup silently diverges from the rest

Broken relative links

A page moved one folder deeper breaks every ../ link built for its old location

The systemic fix: shared partials, one source of truth

Every failure on the previous slide is prevented the same way.

Shared shell file

One header/nav/footer block, reused verbatim, so a nav change is a one-place edit even without a build tool.

Root-relative links

Write links from the site root (/work/) instead of relative paths (../../work/) so moving a page can't silently break its links.

Neither fix requires a framework — both are decisions you make and apply consistently, by hand, across every page.

Before: ad-hoc navigation on a growing site

Three pages, three slightly different navs — a real and common starting state.

```
<!-- about.html -->
<a href="index.html">Home</a>
<a href="work.html">My Work</a>

<!-- work.html -->
<a href="/">Home</a>
<a href="about.html">About Me</a>
<a href="contact.html">Get in touch</a>
```

Three problems visible above: different link text for the same page (“My Work” vs “Work”), a missing Contact link on about.html, and a mixed root-relative/relative style.

After: one shared nav pattern, three pages

Identical markup, identical labels, identical order — pasted, not retyped.

```
<nav aria-label="Primary">
  <ul>
    <li><a href="/">Home</a></li>
    <li><a href="/about/" aria-current="page">About</a></li>
    <li><a href="/work/">Work</a></li>
    <li><a href="/contact/">Contact</a></li>
  </ul>
</nav>
```

This exact block appears on `about.html`, `work.html` and `contact.html` — only the `aria-current` value changes per page.

Testing the IA: can a visitor find it in ≤ 2 clicks?

First-click and tree testing check the structure itself, separate from visual design.

First-click testing

Show a static page or wireframe, ask “where would you click to find X?” — the first click alone predicts most task success.

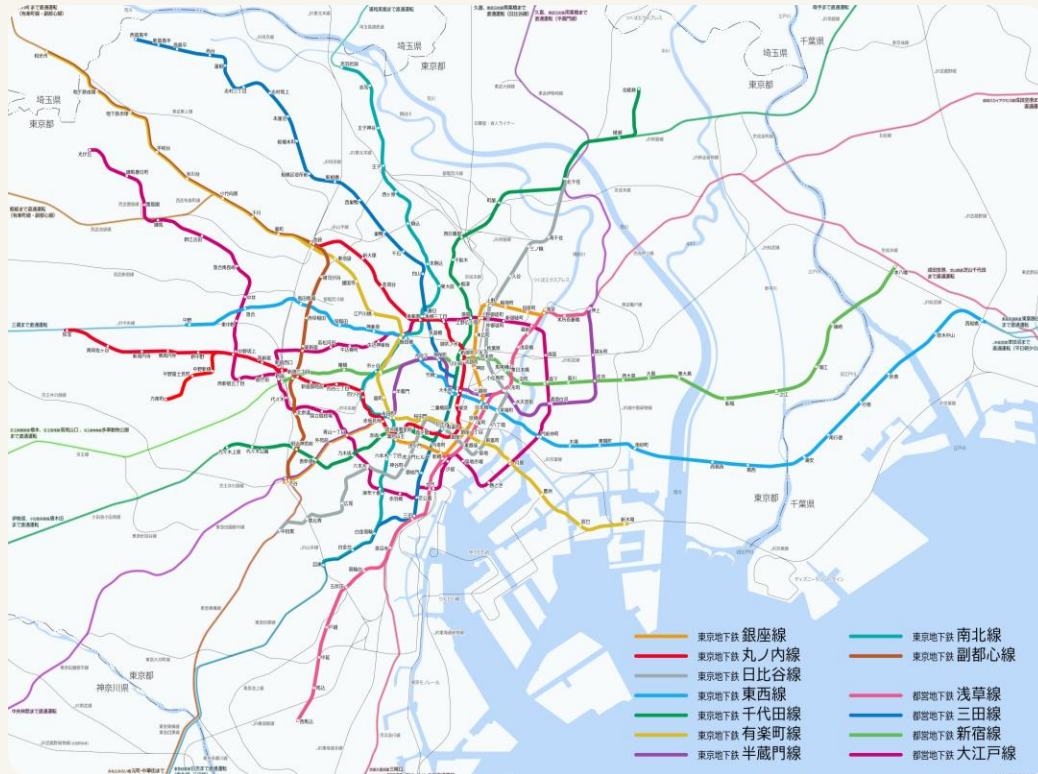
Tree testing

Show only the site's navigation labels as a text list (no visuals) and ask someone to find a target page — isolates the IA from the design entirely.



Reading a real navigation system: two references

Two very different real systems solving the same underlying problem as your site map.



Content organisation as a physical, pre-digital problem

IA did not start with the web — these two systems solved the same grouping problem.



Studio task: sketch, then build

Directly implements Assignment 4's information-architecture requirement.

Sketch

Draw a site map for your Assignment 4 project: choose a topology (hierarchical, flat or hub-and-spoke), list every page, and mark which nav item links to each.

Card-sort check

Ask one classmate to sort your page list into groups without seeing your sketch, then compare the two groupings.

Build

Implement one shared primary-nav pattern, identical in markup and label order, across at least three real HTML pages.

Evidence to submit

The site-map sketch, the three built pages, and a one-paragraph note on the topology you chose and why.

Common IA and navigation mistakes, and the fix

The defects a marker sees most often at this stage of a multi-page build.

Nav grouped by org chart, not by task

Rename groups after the visitor's question, not the team that owns the content

A page only reachable by typing its URL

Click through from home for every inventory item; link every orphan

Nav wording differs page to page

Copy the exact shell block; never retype navigation markup by hand

Detail pages missing a field others have

Check every new page against the written content model before publishing

Recap: this week builds on three earlier weeks

IA and navigation are not a new skill — they assemble three you already have.

Week 2 — HTML and semantic structure

Landmarks (header, nav, main, footer) are the actual elements this week's shell and nav patterns are built from.

Week 6 — Responsive design

A shared shell has to keep working at every screen size — the nav pattern doesn't get a separate design per page.

Week 8 — Accessibility review

Consistent navigation placement, skip links and aria-current are accessibility requirements this week puts into practice.

Recap checklist and what's next

Use this before calling any multi-page structure finished.

- Content inventory lists every page and its purpose.
- Groups are named after visitor tasks, not org structure.
- Every page has at least one inbound nav/footer/breadcrumb link — no orphans.
- Primary nav is identical, in markup and order, on every page.
- URL, nav label and on-page heading agree for every page.
- A tree test or first-click test on the sketch succeeded in ≤ 2 clicks.

Next week: user testing and revision — observing a classmate attempt a real task on the site you just structured.